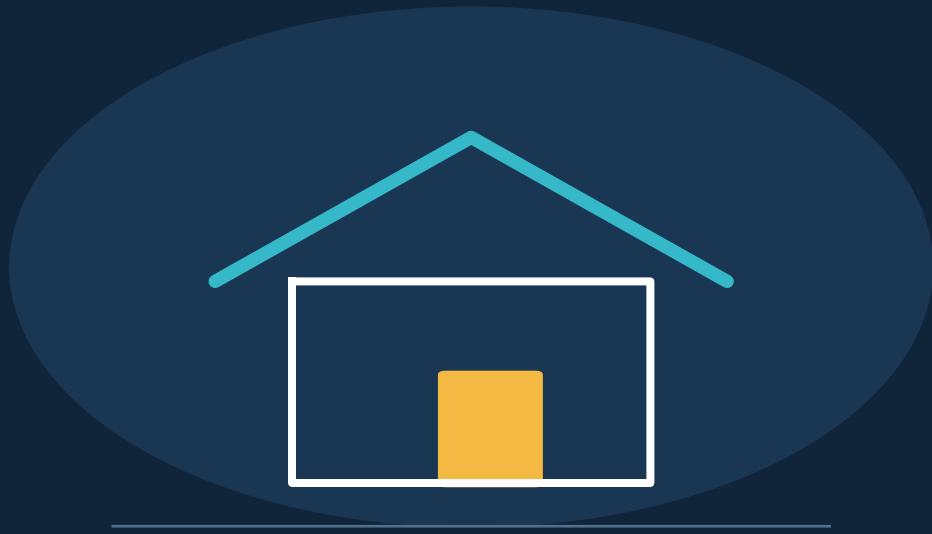
APPLIED MACHINE LEARNING / CALIFORNIA

California Housing Value Prediction



A complete regression workflow: district-level data, model evaluation, experiment tracking, and a live prediction interface.

DATASET

20,640 district records

BEST HELD-OUT MODEL

Random Forest • R^2 0.795

PRODUCT SURFACES

Streamlit • FastAPI • MLflow

Project objective / estimate a district median house value from 9 housing and location features.

The problem & project goal

Translate district-level housing characteristics into a useful, clearly qualified value estimate.

THE QUESTION

Can a model estimate a California district’s median house value from its location, housing stock, and population profile?

PREDICTION TASK

Supervised regression. The target is a continuous dollar value—not a category label.

INTENDED USER FLOW

Enter 9 district attributes in a web form; receive a median-value estimate from the prediction API.



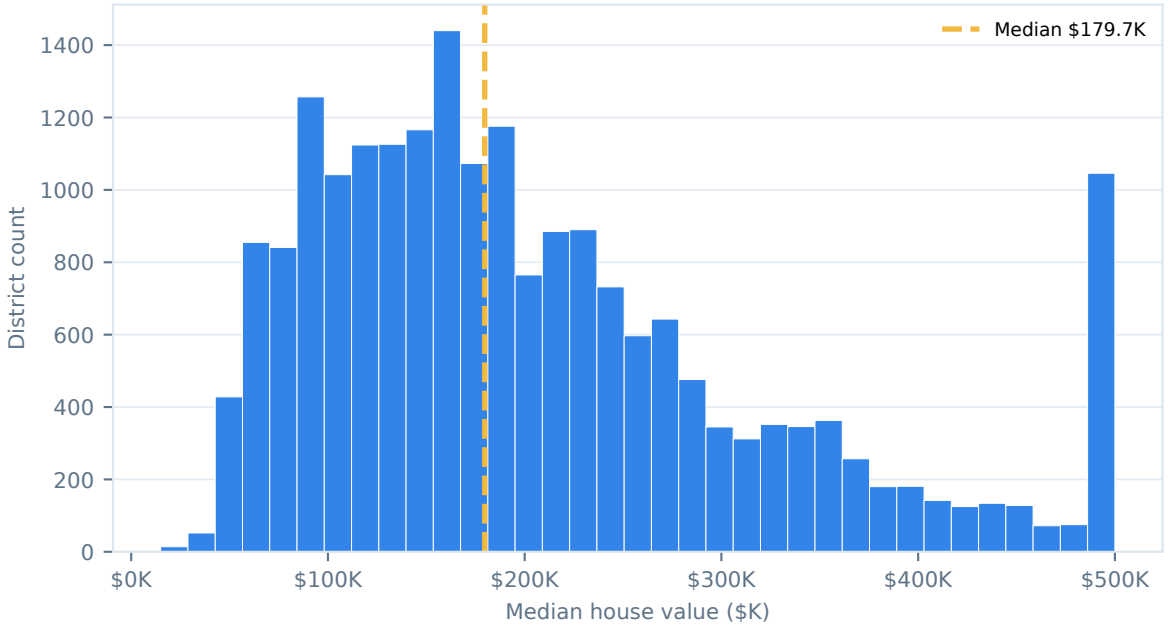
Scope: compare algorithms and operationalize inference. Predictions are estimates, not formal appraisals or lending decisions.

Dataset profile & first look

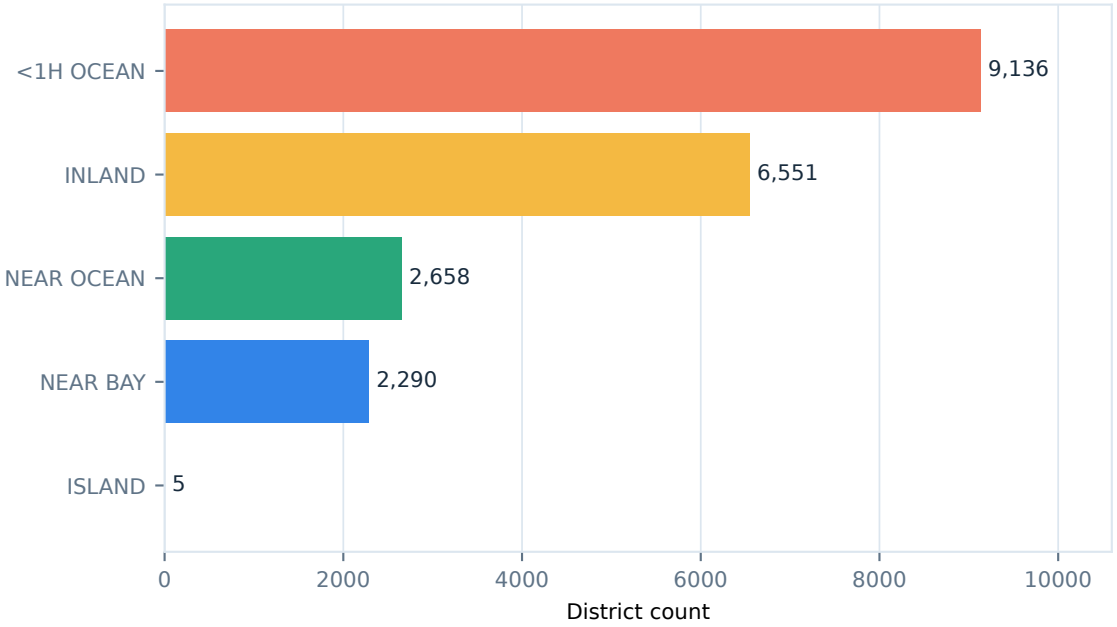
Kaggle California Housing Prices • figures calculated from the downloaded housing.csv.

RECORDS	INPUTS	MISSING	TARGET MEDIAN
20,640	9	207 bedrooms	$179,700

Target distribution / $ thousands



Input categories / ocean proximity



READ THE DISTRIBUTION

Mean 206, 856•*Q*1119,600 • Q3 264, 725•*max*500,001. 965 records equal the maximum, indicating a capped upper tail.

Preparation & price-band weighting

Preprocessing is learned from the training split; regression imbalance is handled through balanced sample weights.

01 SPLIT

80% train
20% test
random_state=42

02 NUMERIC

Median impute
RobustScaler

03 CATEGORY

Most-frequent impute
One-hot encode

04 FIT

Transform test
with train-fitted steps

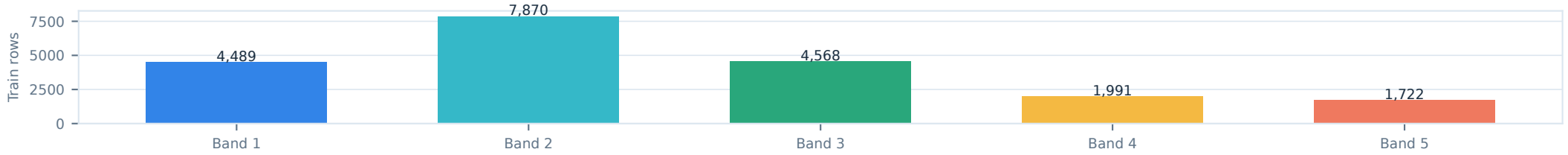
THE IMBALANCE CHALLENGE

House value is continuous. Classification class_weight is not the right direct tool for this regression target.

PROJECT APPROACH

Cut training targets into 5 price bands and compute balanced sample weights; pass weights where supported.

Fixed-width target bands on training data • weighting reduces dominance by common price ranges; it does not create new examples.



Model development

Four complementary regressors share the same prepared feature matrix and held-out test set.

LINEAR REGRESSION

Interpretable baseline
No non-linear interactions by design

RANDOM FOREST

100 trees • max depth 20
min leaf 5 • min split 10

GRADIENT BOOSTING

200 estimators • learning rate 0.05
depth 2 • subsample 0.8

MLP NEURAL NETWORK

Hidden layers 32 → 16 • alpha 0.01
early stopping • 15% validation

Consistent evaluation: seeded 80/20 split; training-only preprocessing; R^2, RMSE, and MAE.

OVERFITTING CONTROL & CHECKS

CONTROLS

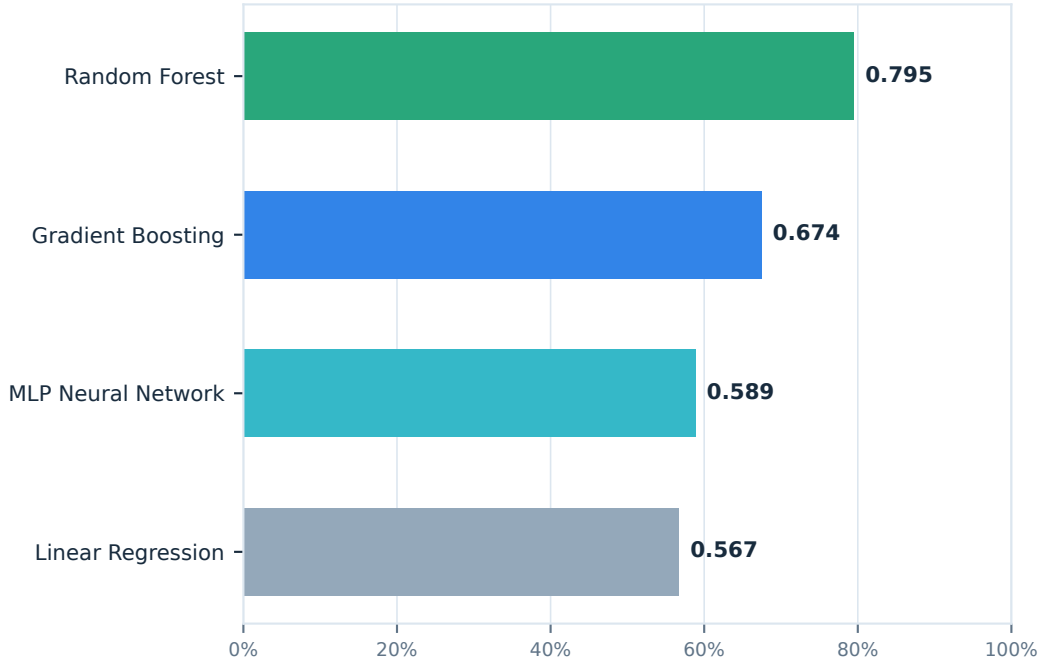
Tree depth / leaf constraints; subsampling; MLP regularization and early stopping.

DIAGNOSTIC

Compare training vs test scores. A large training advantage signals weaker generalization.

Results: test-set comparison

Held-out metrics recalculated from the saved models and the same seeded split. Lower RMSE / MAE is better.



TEST ERROR / USD

Random Forest

RMSE 51,839 *MAE* 34,583

Gradient Boosting

RMSE 65,309 *MAE* 49,541

MLP Neural Network

RMSE 73,420 *MAE* 55,638

Linear Regression

RMSE 75,320 *MAE* 58,255

R^2 compares performance with predicting the test-set mean; 0.795 means the model explains about 79.5% of test-target variance.

BEST TEST RESULT • RANDOM FOREST

R^2 0.795 | RMSE $51,839 | MAE $34,583. On average, predictions miss by about $34.6K in absolute value on this test split.

From training to a prediction

The project keeps the fitted transformation and model together, then serves requests through a validated API.

STREAMLIT

User enters 9 features
Form sends JSON

FASTAPI

Pydantic validation
POST /predict

INFERENCE

Saved preprocessor
+ selected regressor

RESULT

Predicted value
returned in USD

TRACKING & ARTIFACTS

MLflow logs parameters, train/test metrics, plots, and registered model versions. Joblib saves the local bundle.

CONFIGURATION DETAIL

The API currently defaults to Gradient Boosting.
Random Forest had the best test score; select it explicitly with HOUSING_MODEL_NAME to serve the winner.

CONFIGURED LOCAL ENDPOINTS

STREAMLIT	MLFLOW	FASTAPI
8300	8100	8200 host / 8800 container

Conclusion, limitations & next steps

What the current results support—and what should be addressed before real-world use.

WHAT WORKED

Random Forest leads the comparison on the held-out split; the full workflow is packaged for API inference.

WHAT TO WATCH

Train R^2 0.910 vs test R^2 0.795: a 0.115 gap.
Good performance, with room to improve generalization.

LIMITATIONS

- Source target is capped at $500,001; high-end prices are under-resolved.
- One random holdout split is not a substitute for repeated cross-validation.
- Dataset is historical district-level data; current market use needs newer data and drift checks.
- Aggregate errors can hide location- or price-band-specific weaknesses.

BOTTOM LINE A strong project prototype for comparative district-level estimates—not a certified valuation tool.

NEXT ITERATION

01	Cross-validate & tune
02	Audit residuals by geography / price band
03	Refresh data; validate capped-target behavior
04	Set API default to the evaluated model